claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-a4ca47e92978c3c69.jsonl`
- SHA-256: `931a2f8ec0840dd303a8394379e8d18ea0d7d3c77323b790937a371488cdacce`
- Source modified: `2026-06-22T09:14:02+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d9866476-800`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:12:06.304Z)

```
You are mapping the badminton-highlight-indexer repo, checked out at origin/master under
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files
under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal
is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short
golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw
structured data, not prose for a human.

ADVERSARIALLY VERIFY this claim ? try to REFUTE it with concrete code evidence under
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e; default to
"refuted"/"uncertain" if the evidence isn't clearly there. Claim:
""Number of reels" maps to a single specific countable integer artifact in the pipeline output
(name it precisely and where to read it)."

Context from the mapping phase (may be wrong ? re-check against code):
[
 {
  "summary": "\nMapped the reel-production path end-to-end in the badminton-highlight-indexer.
The regression test signal is: **COUNT OF PLAY_SEGMENTS ROWS in the SQLite database per video
after segmentation + filtering**. One stitched output reel (ONE .mp4 file per /api/compile call)
is created from N filtered play_segments. \n\nThe PIPELINE FLOW:\n1. /api/process (or CLI
--video-path) ? _execute_processing ? video_id = MD5(video_file)\n2. Segmenter.process_video ?
detects rally intervals ? inserts rows into play_segments table\n3. /api/compile receives
segment_ids ? queries play_segments ? applies stitching.min_shot_count filter ? calls
VideoCompiler.compile_highlights with (start_time, end_time) pairs\n4. VideoCompiler trims each
segment (with begin/end padding), re-encodes with optional watermark, ffmpeg concat demuxer
stitches N trimmed clips ? ONE .mp4 reel output file\n\n**Regression signal:** `SELECT COUNT(*)
FROM play_segments WHERE video_id = ?` after segmentation completes. This is stable,
deterministic, and queryable headlessly via the database.\n",
  "key_facts": [
   "video_id = compute_video_id(local_path) = MD5 hash
(backend/utils/hashing.py:compute_video_id)",
   "play_segments table schema: id (AUTOINCREMENT), video_id, start_time, end_time, duration,
server, receiver, metadata (JSON). Rows inserted by segmenter via db.add_play_segment().",
   "Segmentation path: main.py::_execute_processing (line 439) ? segmenter.process_video() ?
returns list of rally dicts ? each inserted as play_segments row via
db_instance.add_play_segment()",
   "Compilation path: /api/compile (line 705) ? query_play_segments(video_id, {}) ? filter by
segment_ids + stitching.min_shot_count (from metadata['shot_count']) ? extract time_pairs
[(start, end), ...] ? VideoCompiler.compile_highlights(video_path, time_pairs,
output_filename)",
   "VideoCompiler.compile_highlights (compiler.py line 305): normalizes timestamps ? merges
overlapping ranges ? for each segment: _trim_one_segment() (line 192) ffmpeg -ss -to re-encodes
with padding + watermark ? ffmpeg concat demuxer stitches clips ? ONE output .mp4 file",
   "Stitching config (backend/config/models.py line 255): begin_padding=0.5s, end_padding=0.5s
```

(per-segment), min_shot_count=1 (metadata filter), watermark enabled by default",
   "Output: ONE .mp4 file per /api/compile call at output_dir/{output_filename}. Stored flat (no video_id subdirs) ? /api/reels lists all .mp4 files in output_dir.",
   "Regression signal (stable, headless, deterministic): `SELECT COUNT(*) FROM play_segments WHERE video_id = <video_id>` after _execute_processing completes (after destroy-AFTER-confirm finalization at main.py line 536).",
   "Database path: global_config.output.output_dir / global_config.output.db_name (default: output/rally_indexer.db)",
   "CLI single-shot headless mode: `indexer --video-path <path> [--segmenter <name>] [--output-dir <dir>]` (no web UI, exits after segmentation) ? main.py line 997-1075"
  ],
  "recommendation": "\nFor the NIGHTLY E2E REGRESSION on 2?3 golden videos:\n\n**REGRESSION SIGNAL:** Count of play_segments rows in the output DB for each golden video. Expected counts should be hardcoded from HEAD baseline.\n\n**EXACT HEADLESS COMMANDS:**\n\n```bash\n# 1. Run segmentation (single-shot CLI mode, no server, no UI):\nindexer --video-path /path/to/golden_video.mp4 \\\n   --segmenter motion \\\n   --output-dir output/nightly_e2e\n\n# 2. Query the segment count (read-only DB inspection):\npython -c \"\n from backend.infrastructure.database import Database\n db = Database('output/nightly_e2e/rally_indexer.db')\n from backend.utils.hashing import compute_video_id\n video_id = compute_video_id('/path/to/golden_video.mp4')\n with db._connect() as conn:\n   count = conn.cursor().execute(\n   'SELECT COUNT(*) FROM play_segments WHERE video_id = ?', \n   (video_id,)\n   ).fetchone()[0]\n print(count)\n\"\n\n# 3. Assert count == baseline for each video:\n#   Fail (exit 1) if ANY video's count deviates\n```\n\n**NIGHTLY REGRESSION HARNESS STRUCTURE:**\n\n1. **Checkpointed baseline:** `eval_baselines/nightly_e2e_baselines.json` (commit into repo)\n   ```json\n   {\n \"golden_videos\": [\n   {\"name\": \"video1.mp4\", \"md5\": \"<video_id>\", \"expected_segment_count\": 3},\n   {\"name\": \"video2.mp4\", \"md5\": \"<video_id>\", \"expected_segment_count\": 5},\n   {\"name\": \"video3.mp4\", \"md5\": \"<video_id>\", \"expected_segment_count\": 4}\n   ],\n   \"generator\": \"HEAD <commit>\",\n \"timestamp\": \"2026-06-22T00:00:00Z\"\n   }\n   ```\n\n2. **Nightly runner script:** Run headlessly on each golden video ? count play_segments ? compare to baseline ? exit 1 on mismatch\n\n3. **STITCHING PIPELINE VERIFICATION (optional, post-segmentation):**\n   - After segment count passes, optionally call /api/compile via Python to produce a test reel\n   - Assert output .mp4 file exists and has size > 0\n   - (Stitching is more complex ? depends on segment timing + ffmpeg + watermark rendering; segment count is the stable proxy)\n\n**EXACT INTEGER TO TRACK:**\n- **Primary:** `COUNT(*) FROM play_segments WHERE video_id = <video_id>` after segmentation\n- **Secondary (optional):** COUNT of .mp4 files in output_dir that match the compile request (should be 1 per /api/compile call)\n\n**CONFIG FOR GOLDEN VIDEOS:**\n- Use `--segmenter motion` (offline, no API key, no GPU ? reproducible)\n- Default padding/watermark (stitching.begin_padding=0.5, end_padding=0.5)\n- No cloud storage (use local paths only)\n",
  "confidence": "high",
  "code_refs": [
   "backend/main.py:439::_execute_processing",
   "backend/main.py:559::process_video",
   "backend/main.py:705::compile_highlights",
   "backend/main.py:997-1075",
   "backend/infrastructure/database.py:24::Database",
   "backend/infrastructure/database.py:564::query_play_segments",
   "backend/infrastructure/database.py:459::add_play_segment",
   "backend/pipelin

2. assistant (2026-06-22T09:12:09.018Z)

I'll adversarially verify this claim by examining the actual codebase to confirm whether "number of reels" maps to a single specific countable integer artifact.

3. user (2026-06-22T09:12:10.782Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-e2e'

4. assistant (2026-06-22T09:12:13.103Z)

Let me start by finding the repository structure and examining the actual pipeline code.

5. user (2026-06-22T09:12:14.707Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/main.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/stitcher/compiler.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_compiler.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_database.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/backend/infrastructure/database.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/backend/main.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/backend/pipeline/stitcher/compiler.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/tests/test_compiler.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/tests/test_database.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/boxhill17/backend/infrastructure/database.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/backend/main.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/boxhill17/backend/pipeline/stitcher/compiler.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/boxhill17/tests/test_compiler.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/boxhill17/tests/test_database.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-
doc/backend/infrastructure/database.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-doc/backend/main.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-
doc/backend/pipeline/stitcher/compiler.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-
doc/tests/test_compiler.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-
doc/tests/test_database.py
```

6. assistant (2026-06-22T09:12:16.802Z)

I see multiple worktrees. Let me check if nightly-e2e exists:

7. user (2026-06-22T09:12:17.360Z)

```
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 nightly-e2e
```

8. assistant (2026-06-22T09:12:19.460Z)

Now let me check the structure of that worktree:

9. user (2026-06-22T09:12:20.243Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/infrastructure/database.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/main.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/stitcher/compiler.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/worker/__main__.py
C:/Users/avidu/Projects/badminton-highlight-
```

indexer/.claude/worktrees/nightly-e2e/tests/test_compiler.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_database.py

10. assistant (2026-06-22T09:12:22.771Z)

Good. Now let me examine the key files to verify the claim. Let me start with understanding what "number of reels" actually means in the output:

11. user (2026-06-22T09:12:23.530Z)

```
1       import os
2       import sys
3       import argparse
4       import json
5       import logging
6       import shutil
7       import socket
8       import threading
9       import uuid
10      import webbrowser
11      from importlib.metadata import PackageNotFoundError, version as _pkg_version
12      from urllib.parse import quote
13      import uvicorn
14      from dotenv import load_dotenv
15
16      # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
17      # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
18      # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
19      # stdlib-only import ? keeps this pre-pydantic line light.
20      from backend.utils.app_home import env_file_to_load, resolve_output_dir
21
22      load_dotenv(env_file_to_load())
23
24      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
25      logging.basicConfig(
26          level=logging.INFO,
27          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
28          datefmt='%H:%M:%S'
29      )
30      logger = logging.getLogger(__name__)
31      from fastapi import FastAPI, HTTPException, Request
32      from fastapi.middleware.cors import CORSMiddleware
33      from starlette.middleware.trustedhost import TrustedHostMiddleware
34      from pydantic import BaseModel
35      from typing import List, Dict, Any, Optional
36
37      from fastapi.staticfiles import StaticFiles
38      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
39      from backend.pipeline.validators.base import registry
40      from backend.infrastructure.database import Database
41      from backend.pipeline.segmenters.base import segmenter_registry
42      from backend.pipeline.stitcher.compiler import VideoCompiler
43      from backend.pipeline.stitcher.watermark_i18n import localize_watermark
44      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
45      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
46      from backend.utils.redact import redact_secrets  # P0b: mask secret-shaped fields in
/api/config
47      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
48      from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
```

```
resolver
49      from backend.config import (  # new typed config
50          load_config as load_app_config,
51          write_light_default_config,  # P2b: batteries-included first-run config seeding
52          AppConfig,
53          StitchingConfig,
54          enforce_startup_checks,
55          StartupCheckError,
56      )
57      from backend.results import Failure  # standardized error/result contract
58      from backend.reporting import emit_indexer_report
59      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
60      from backend import billing  # M8: per-job cost ledger (USD internal / INR display)
61      from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-
OFF)
62
63      app = FastAPI(title="Sports Highlight Indexer API")
64
65
66      def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
67          """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
68          default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
69          does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
70          src = os.environ if env is None else env
71          raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
72          origins = [o.strip() for o in raw.split(",") if o.strip()]
73          return origins or ["*"]
74
75
76      # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
77      # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
78      # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
79      # lock it to its origin; the middleware is fixed at startup.
80      app.add_middleware(
81          CORSMiddleware,
82          allow_origins=_cors_origins_from_env(),
83          allow_credentials=False,
84          allow_methods=["*"],
85          allow_headers=["*"],
86      )
87
88
89      def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
90          """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
91          web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
92          the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
93          Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
94          Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
95          ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
96          deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
97          src = os.environ if env is None else env
```

```
98          raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
99          hosts = [h.strip() for h in raw.split(",") if h.strip()]
100         # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
```

12. assistant (2026-06-22T09:12:25.795Z)

Let me search for the /api/reels endpoint and understand how reels are counted:

13. user (2026-06-22T09:12:27.614Z)

```
Found 3 files
.claude\worktrees\nightly-e2e\backend\utils\workspace.py
.claude\worktrees\nightly-e2e\backend\ui\assets\index-GSXqL6iY.js
.claude\worktrees\nightly-e2e\backend\main.py
```

14. assistant (2026-06-22T09:12:30.375Z)

Let me search more specifically for the reels endpoint:

15. user (2026-06-22T09:12:31.173Z)

```
381:def _reels_dir() -> str:
388:@app.get("/api/reels")
389:def list_reels():
390:    """List compiled highlight reels present in the output dir, newest first, with a
download URL.
392:    NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
396:    base = _reels_dir()
397:    reels = []
401:        return {"reels": []}
409:                reels.append({
413:                    "download_url": f"/api/reels/{quote(e.name)}",
415:    reels.sort(key=lambda r: r["modified_at"], reverse=True)
416:    return {"reels": reels}
419:@app.get("/api/reels/{filename}")
421:    """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
427:    base = _reels_dir()
```

16. assistant (2026-06-22T09:12:33.174Z)

Now let me look at the /api/compile endpoint to understand how reels are generated:

17. user (2026-06-22T09:12:34.125Z)

```
705:@app.post("/api/compile")
706:def compile_highlights(req: CompileRequest):
```

18. assistant (2026-06-22T09:12:36.617Z)

Let me read the compile endpoint:

19. user (2026-06-22T09:12:37.631Z)

```
705     @app.post("/api/compile")
706     def compile_highlights(req: CompileRequest):
707         video = db_instance.get_video(req.video_id)
708         if not video:
709             raise HTTPException(status_code=404, detail="Indexed video record not found.")
710
711         # Retrieve matching play segments from db
```

```
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "

f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
        # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
        # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        localized_watermark = localize_watermark(stitching.watermark, req.locale)
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
watermark=localized_watermark)
        try:
            # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
            # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
            # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
            # for a bucket URI (the same seam the process path uses, _execute_processing
```

```
main.py:159).
763             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
764             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
765             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
766             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
767             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
768             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
769             return {"status": "success", "filepath": out_path, "download_url": download_url}
770         except Exception as e:
771             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
772
773     # --- CLI Debug Utility & Orchestration ---
774     def run_cli_diagnose_clip(video_path: str, timestamp: float):
775         """CLI debugging utility to examine segment properties around a timestamp."""
776         print("=" * 60)
777         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
778         print("=" * 60)
779
780         cfg = load_config()  # legacy wrapper still works, returns dict for now
781
782         # 1. Validation check diagnostics
783         print("[DIAGNOSTIC] Running validation framework...")
784         results = registry.run_all(video_path, cfg)
785         for r in results:
786             status_symbol = "[PASS]" if r.passed else "[FAIL]"
787             print(f"  {status_symbol} {r.validator_name}: {r.message}")
788             if r.details:
789                 print(f"         Details: {r.details}")
790
791         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
792         print("-" * 60)
793         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
794         import cv2
795         cap = cv2.VideoCapture(video_path)
796         if not cap.isOpened():
797             print("[FAIL] OpenCV could not open video.")
798             return
799
800         fps = cap.get(cv2.CAP_PROP_FPS)
801         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
802
803         start_frame = max(0, int((timestamp - 3.0) * fps))
804         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
```

20. assistant (2026-06-22T09:12:40.682Z)

Now let me check what the compile_highlights method returns and how "reel" output files are counted:

21. user (2026-06-22T09:12:41.690Z)

```
1       import os
2       import subprocess
3       import tempfile
4       import logging
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin  # P2a: single ffmpeg/ffprobe
resolver
```

```

    logger = logging.getLogger(__name__)

    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the overlay
    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows, where
    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives at
    # backend/pipeline/stitcher/, so parents[2] == backend/.
    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"

    class VideoCompiler:
        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
                     watermark: Optional[Any] = None):
            self.output_dir = output_dir
            self.end_padding = end_padding
            self.begin_padding = begin_padding
            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
            # normalized to a dict (or None) so this module stays config-package-agnostic.
            self.watermark = self._normalize_watermark(watermark)
            os.makedirs(output_dir, exist_ok=True)

        @staticmethod
        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
            if watermark is None:
                return None
            if hasattr(watermark, "model_dump"):
                return watermark.model_dump()
            if isinstance(watermark, dict):
                return dict(watermark)
            return None

        @staticmethod
        def _ff_quote(path: str) -> str:
            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
            slashes (Windows-friendly), escaped single quotes, AND escaped colons.

            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
            option parser treats ':' as the option separator, so a Windows drive-letter
            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
            the encode fails with "Invalid argument". This silently disabled the
            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")

        @staticmethod
        def _parse_time(val: Union[int, float, str]) -> float:
            """Normalizes a timestamp value to float seconds.
            Handles: float/int (pass-through), plain numeric strings ("12.4"),
            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
            This mirrors the parse_time logic used in gemini.py to ensure
            the stitcher can always consume whatever format the indexer produced."""
            if isinstance(val, (int, float)):
                return float(val)
            if isinstance(val, str):
                val = val.strip()
                if ":" in val:
                    # Handle MM:SS, HH:MM:SS, etc.
                    parts = val.split(":")
                    parts.reverse()
                    total = 0.0
                    for i, p in enumerate(parts):
```

```
                        try:
                            total += float(p) * (60 ** i)
                        except ValueError:
                            pass
                    return total
                try:
                    return float(val)
                except ValueError:
                    logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
                    return 0.0
            logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
            return 0.0
```

22. assistant (2026-06-22T09:12:44.499Z)

Let me find the compile_highlights method:

23. user (2026-06-22T09:12:45.874Z)

```
305:    def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
```

24. user (2026-06-22T09:12:49.258Z)

```
305         def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
306             """
307             Extracts selected segments from a raw video and stitches them together
308             into a seamless highlights file, applying end_padding to prevent abrupt endings.
309             """
310             if not segments:
311                 raise ValueError("No highlight segments provided to compile.")
312
313             # Never stitch the same footage twice: collapse overlapping/duplicate ranges
314             # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
315             original_count = len(segments)
316             segments = self._merge_overlapping(segments)
317             if len(segments) < original_count:
318                 logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
319                             original_count - len(segments), original_count, len(segments))
320             if not segments:
321                 raise ValueError("No valid (start < end) highlight segments after merge.")
322
323             if not os.path.exists(raw_video_path):
324                 raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
325
326             # Defense-in-depth: never let the output name escape output_dir (the API also
327             # validates this at the request boundary).
328             output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
329
330             # Probe video duration so we can detect out-of-range segments
331             video_duration = self._get_video_duration(raw_video_path)
332             if video_duration > 0:
333                 logger.info(f"Source video duration: {video_duration:.2f}s")
334             else:
335                 logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
336
337             # We will create temporary clips and stitch them using FFmpeg concat demuxer
338             temp_clips = []
```

```
            skipped_segments = []

            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
                # Build the (optional) watermark filtergraph once; it is applied during
every
                # per-clip re-encode so the mark is baked into each clip identically.
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
                    logger.info("[watermark] applying branding overlay to each clip.")
                for idx, (raw_start, raw_end) in enumerate(segments):
                    clip_path, skip_record = self._trim_one_segment(
                        idx, raw_start, raw_end, segments, video_duration,
                        tmp_dir, raw_video_path, watermark_filter,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
                        skipped_segments.append(skip_record)

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
                # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
                # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
                with open(list_file_path, "w", encoding="utf-8") as f:
                    for clip in temp_clips:
                        # Escape single quotes in path
                        safe_path = clip.replace("'", "'\\''")
                        f.write(f"file '{safe_path}'\n")

                logger.info("Stitching clips together...")
                # Stitch them together without re-encoding (since they were encoded with
identical settings)
                concat_cmd = [
                    ffmpeg_bin(), "-y",
                    "-f", "concat",
                    "-safe", "0",
                    "-i", list_file_path,
                    "-c", "copy",
                    output_path
                ]
```

```
396
397                 try:
398                     subprocess.run(concat_cmd, capture_output=True, check=True)
399
400                     # Final verification
401                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
402                         raise RuntimeError(
403                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
404                             f"but output file is missing or empty: {output_path}"
405                         )
406
407                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
408                     logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
409                     return output_path
410                 except subprocess.CalledProcessError as e:
411                     stderr_msg = e.stderr.decode(errors='replace').strip()
412                     raise RuntimeError(
413                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
414                         f"  Command: {' '.join(concat_cmd)}\n"
415                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
416                     ) from e
417
```

25. assistant (2026-06-22T09:12:52.541Z)

Now let me examine the database schema to understand play_segments:

26. user (2026-06-22T09:12:53.736Z)

```
1       import os
2       import shutil
3       import sqlite3
4       import json
5       import logging
6       from contextlib import contextmanager
7       from typing import List, Dict, Any, Optional, Tuple
8
9       from backend import billing  # pure cost math (no backend imports ? no cycle)
10
11      logger = logging.getLogger(__name__)
12
13      # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14      # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15      # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16      # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17      # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18      SCHEMA_VERSION = 7
19
20
21      class SchemaTooNewError(RuntimeError):
22          """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24      class Database:
25          def __init__(self, db_path: str):
26              self.db_path = db_path
27              self._init_db()
28
29          @contextmanager
30          def _connect(self):
```

```
            """Transaction scope that also CLOSES the connection.

            ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
            never closes, so every call leaked its connection to GC (lingering file
            handles on the WAL sidecars under Windows). Internal methods use this.
            """
            conn = self._get_connection()
            try:
                with conn:
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
            """Run ONE bool-returning single-statement write inside a connection scope.

            Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
            the bool-returning single-statement writers. ``error_msg`` is the caller's exact
            per-method message (already formatted with its own ids) ? logged verbatim on a
            swallowed write fault so each writer keeps its specific log text. Returns True
on
            success, False (after logging ``error_msg``) on any exception."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(sql, params)
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"{error_msg}: {e}")
                return False

        @staticmethod
        def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
            """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.

            A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
            transaction, so an interrupted v3/v4 block can leave the column added while
            ``user_version`` stays un-bumped ? and the next open would re-run the same
            ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
            future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
            ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
            ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
            """
            try:
                cursor.execute(ddl)
            except sqlite3.OperationalError as exc:
```

```
91                 if "duplicate column name" not in str(exc).lower():
92                     raise
93
94         def _init_db(self):
95             """Initializes tables for videos, play_segments, and events."""
96             with self._connect() as conn:
97                 cursor = conn.cursor()
98
99                 self._create_base_tables(cursor)
100
101                # Versioned migrations live here. PRAGMA user_version is the schema
102                # contract ? bump it for every change and add an ordered `if version < N`
103                # block below. Tests assert the expected version, so accidental drift fails
CI.
104                version = cursor.execute("PRAGMA user_version").fetchone()[0]
105
106                # P2d upgrade-safety (no-op for a fresh DB or one already at
SCHEMA_VERSION):
107                # (1) DOWNGRADE GUARD ? a DB from a NEWER app (version > what we know) has
schema this
108                #     binary can't reason about; opening it could corrupt data, so refuse
loudly.
109                if version > SCHEMA_VERSION:
110                    raise SchemaTooNewError(
111                        f"This database (schema v{version}) was created by a newer version
of the app "
112                        f"than this one (knows up to v{SCHEMA_VERSION}). Upgrade the app, or
point it at "
113                        f"a different data directory (RALLY_APP_HOME). DB: {self.db_path}")
114                # (2) BACKUP-BEFORE-MIGRATE ? an EXISTING older DB is about to be upgraded
in place; a
115                #     failed migration could corrupt it, so snapshot it first (best-effort:
WARN, never
116                #     block the upgrade). version 0 = a brand-new DB ? nothing to back up.
117                if 0 < version < SCHEMA_VERSION:
118                    self._backup_before_migrate(conn, version)
119
120                if version < 1:
121                    self._migrate_to_v1(cursor)
122
123                if version < 2:
124                    self._migrate_to_v2(cursor)
125
126                if version < 3:
127                    self._migrate_to_v3(cursor)
128
129                if version < 4:
130                    self._migrate_to_v4(cursor)
131
132                if version < 5:
133                    self._migrate_to_v5(cursor)
134
135                if version < 6:
136                    self._migrate_to_v6(cursor)
137
138                if version < 7:
139                    self._migrate_to_v7(cursor)
140                # future: if version < 8: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 8
141
142                conn.commit()
143
144        def _backup_before_migrate(self, conn: sqlite3.Connection, from_version: int) ->
None:
145            """Snapshot the DB to ``<db>.bak-v<from_version>`` before an upgrade migration
```

```
(P2d).
146
147             Checkpoints the WAL into the main file first so the copy is complete. Best-
effort: a backup
148             failure WARNS but does NOT block the upgrade (the ladder is itself crash-safe;
refusing to
149             start because a backup couldn't be written would be worse than proceeding
without one)."""
150             backup = f"{self.db_path}.bak-v{from_version}"
```

27. assistant (2026-06-22T09:12:55.791Z)

Let me find the play_segments table schema:

28. user (2026-06-22T09:12:57.312Z)

```
95:        """Initializes tables for videos, play_segments, and events."""
172:        """Create the videos / play_segments / events base tables (idempotent)."""
186:            CREATE TABLE IF NOT EXISTS play_segments (
208:                FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
233:                FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON DELETE
SET NULL
432:            p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id =
?",
441:        """Delete play/candidate segments by id range (events cascade from play_segments).
450:                cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
452:                cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
464:                    "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
564:    def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
572:        query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
```

29. user (2026-06-22T09:12:59.947Z)

```
186                 CREATE TABLE IF NOT EXISTS play_segments (
187                     id INTEGER PRIMARY KEY AUTOINCREMENT,
188                     video_id TEXT NOT NULL,
189                     start_time REAL NOT NULL,
190                     end_time REAL NOT NULL,
191                     duration REAL NOT NULL,
192                     server TEXT,
193                     receiver TEXT,
194                     metadata TEXT,
195                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
196                 )
197             """)
198
199             # Events Table
200             cursor.execute("""
201                 CREATE TABLE IF NOT EXISTS events (
202                     id INTEGER PRIMARY KEY AUTOINCREMENT,
203                     segment_id INTEGER NOT NULL,
204                     timestamp REAL NOT NULL,
205                     type TEXT NOT NULL,
206                     player TEXT,
207                     details TEXT,
208                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
209                 )
210             """)
211
212         def _migrate_to_v1(self, cursor):
```

```
213             # v1: raw candidate detections (pre-confirmation), detector-agnostic.
214             # Common fields are columns; detector-specific metrics go in `stats` JSON,
215             # so a new segmenter reuses this table by setting its own `detector`/`stats`.
216             cursor.execute("""
217                 CREATE TABLE IF NOT EXISTS candidate_segments (
218                     id INTEGER PRIMARY KEY AUTOINCREMENT,
219                     video_id TEXT NOT NULL,
220                     detector TEXT NOT NULL,
221                     chunk_index INTEGER,
222                     start_time REAL NOT NULL,
223                     end_time REAL NOT NULL,
224                     duration REAL NOT NULL,
225                     confidence REAL,
226                     stats TEXT,
227                     detection_params TEXT,
228                     confirmed_segment_id INTEGER,
229                     human_label TEXT,
230                     notes TEXT,
231                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
232                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
233                     FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
234                 )
235             """)
```

30. assistant (2026-06-22T09:13:01.994Z)

Now let me check if "reel" in the regression test context refers to the actual output files:

31. user (2026-06-22T09:13:03.213Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/per_user_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/nightly_baseline.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/run_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.ps1
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/register_nightly_task.ps1
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_eval_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_nightly_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_per_user_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_served_gate_a_regression.py
```

32. assistant (2026-06-22T09:13:05.258Z)

Perfect! Let me examine the nightly regression script and the baseline:

33. user (2026-06-22T09:13:06.343Z)

```
1       """Nightly rally-quality regression tripwire.
2
3       Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
```

```
4       seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5       the headline metrics against a FROZEN baseline
(``eval_baselines/nightly_baseline.json``).
6       Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7       over-segmentation-guard regression.
8
9       This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10      + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11      night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12      does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13      no WASB re-extraction (those are cached and unchanged night to night).
14
15      Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17      * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18        to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19        SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20      * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21        ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22        the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23      * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24        permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25        win must keep standing its ground, not be taken at launch-time value).
26
27      Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28      outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29      baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30      whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31      has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32      ``scripts/nightly_regression.ps1``.)
33
34      Usage
35      -----
36      Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38          python scripts/nightly_regression.py --update-baseline
39
40      Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42          python scripts/nightly_regression.py
43
44      Exit codes
45      ----------
46      * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47      * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
```

```
per-video
48        over-seg guard increased). Takes precedence over a stale baseline.
49      * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50      * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51      * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52        but no regression was found on the comparable intersection.
53      """
54      from __future__ import annotations
55
56      import argparse
57      import datetime as _dt
58      import json
59      import os
60      import sys
61      from dataclasses import dataclass, field, replace
62      from typing import Any, Dict, List, Optional, Tuple
63
64      # ---------------------------------------------------------------------------- #
65      # Paths (resolved relative to the repo root, so the script is cwd-robust).
66      # ---------------------------------------------------------------------------- #
67      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68      if REPO_ROOT not in sys.path:
69          sys.path.insert(0, REPO_ROOT)  # importable as `python
scripts/nightly_regression.py` from anywhere
70      DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71      DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72      DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73      DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74      DEFAULT_IOU = 0.5
75
76      # ---------------------------------------------------------------------------- #
77      # What we gate on (a tripwire ? focused, not every metric).
78      #   * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
79      #   * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
80      #   * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
81      #     (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82      # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83      # ---------------------------------------------------------------------------- #
84      GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85      GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86      PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88      #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89      AGG_SNAPSHOT: Tuple[str, ...] = (
90          "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91          "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92      )
93      #: Per-video metrics snapshotted into the baseline.
94      PV_SNAPSHOT: Tuple[str, ...] = (
95          "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96      )
97
98      DEFAULT_METRIC_TOL = 0.005   # F1-like drop that counts as a regression (absorbs float
jitter)
99      DEFAULT_RATE_TOL = 0.010     # merge_rate rise that counts as a regression
100     DEFAULT_OVER_SEG_TOL = 0     # per-video split/merge increase allowed (strict)
101
102
103     # ---------------------------------------------------------------------------- #
104     # The two tracked configs.
```

```
    # ---------------------------------------------------------------------- #
    @dataclass(frozen=True)
    class NightlyConfig:
        """A named windowing config the nightly re-scores (preset + net-crossings gate)."""

        name: str
        preset: Any              # backend.eval.windowing.WindowingPreset
        min_crossings: int = 0
        note: str = ""


    def tracked_configs() -> List[NightlyConfig]:
        """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).

        All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
        min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:

        * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
          point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
          DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
          knob.
        * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
          the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
          what we expect?" stays answerable even if a future change moves DEFAULT.
        * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
          the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
          dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
          launch-time value). It is the reference DEFAULT used to be before the flip."""
        from backend.eval.windowing import SERVED

        default = NightlyConfig(
            name="default", preset=SERVED, min_crossings=0,
            note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
        )
        milestone = NightlyConfig(
            name="milestone",
            preset=replace(SERVED, name="milestone", max_window_duration=6.0,
                           inactivity_end=1.5, inactivity_rally_thresh=0.20,
                           inactivity_min_density=0.30),
            min_crossings=0,
            note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
        )
        baseline_off = NightlyConfig(
            name="baseline_off",
            preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
hard_cap=False,
                           inactivity_end=0.0, inactivity_rally_thresh=0.08,
inactivity_min_density=0.34,
                           blob_fallback=False, blob_factor=4.0),
            min_crossings=0,
            note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
        )
```

```
        return [default, milestone, baseline_off]


    # ---------------------------------------------------------------------- #
    # Tolerances bundle.
    # ---------------------------------------------------------------------- #
    @dataclass(frozen=True)
    class Tolerances:
        metric_tol: float = DEFAULT_METRIC_TOL
        rate_tol: float = DEFAULT_RATE_TOL
        over_seg_tol: int = DEFAULT_OVER_SEG_TOL

        def to_dict(self) -> Dict[str, Any]:
            return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
                    "over_seg_tol": self.over_seg_tol}


    # ---------------------------------------------------------------------- #
    # Pure summary + comparison core (no I/O ? unit-testable).
    # ---------------------------------------------------------------------- #
    def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
        """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.

        Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
        which is seeded but irrelevant to the gate) and per-video RAW metric values."""
        agg = eval_result.get("aggregate", {}) or {}
        aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
        per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
        return {
            "n": int(agg.get("n", len(eval_result.get("rows", [])))),
            "preset": eval_result.get("preset"),
            "min_crossings": eval_result.get("min_crossings", 0),
            "iou": eval_result.get("iou", DEFAULT_IOU),
            "aggregate": aggregate,
            "per_video": per_video,
        }


    @dataclass
    class Finding:
        """One comparison outcome line (regression / improvement / stale note)."""

        config: str
        scope: str          # "aggregate" | "per_video" | "config"
        metric: str
        kind: str           # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        video: Optional[str] = None

        @property
        def delta(self) -> Optional[float]:
            if self.baseline is None or self.current is None:
                return None
            return self.current - self.baseline

        def message(self) -> str:
            loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
            if self.kind == "stale":
                return f"[STALE] {self.config}: {self.metric}"
            d = self.delta
```

```
            ds = f"{d:+.4f}" if d is not None else "?"
            b = "?" if self.baseline is None else f"{self.baseline:.4f}"
            c = "?" if self.current is None else f"{self.current:.4f}"
            tag = "REGRESSION" if self.kind == "regression" else "improvement"
            return f"[{tag}] {loc}: {b} ? {c} ({ds})"


    @dataclass
    class ConfigComparison:
        config: str
        status: str = "ok"            # "ok" | "regression" | "stale"
        findings: List[Finding] = field(default_factory=list)
        added_videos: List[str] = field(default_factory=list)
        removed_videos: List[str] = field(default_factory=list)
        aggregate_gated: bool = True  # False when preset/corpus changed ? aggregate not
comparable

        @property
        def regressions(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "regression"]

        @property
        def improvements(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "improvement"]


    def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
        """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
        if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
            return base_preset == cur_preset
        keys = set(base_preset) | set(cur_preset)
        return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")


    def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
                       tols: Tolerances) -> ConfigComparison:
        """Compare one config's current summary to its baseline summary.

        * If the preset definition or the net-crossings gate changed, the whole config is
STALE
          (numbers aren't comparable) ? flag it, skip gating.
        * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
          skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
          INTERSECTION (a code regression on a shared video must still trip the wire).
        """
        cmp = ConfigComparison(config=name)

        # 1) Config-definition drift ? stale, not gateable.
        if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
                or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
            cmp.status = "stale"
            cmp.aggregate_gated = False
            cmp.findings.append(Finding(config=name, scope="config", metric="preset",
kind="stale"))
            return cmp

        base_pv = base_sum.get("per_video", {})
        cur_pv = cur_sum.get("per_video", {})
        base_names, cur_names = set(base_pv), set(cur_pv)
        cmp.added_videos = sorted(cur_names - base_names)
        cmp.removed_videos = sorted(base_names - cur_names)
        corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
```

```
275
276         # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
277         if corpus_changed:
278             cmp.aggregate_gated = False
279             cmp.status = "stale"
280             cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
281         else:
282             base_agg = base_sum.get("aggregate", {})
283             cur_agg = cur_sum.get("aggregate", {})
284             for m in GATED_HIGHER:
285                 b, c = base_agg.get(m), cur_agg.get(m)
286                 if b is None or c is None:
287                     continue
288                 if c < b - tols.metric_tol:
289                     cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
290                 elif c > b + tols.metric_tol:
291                     cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
292             for m in GATED_LOWER:
293                 b, c = base_agg.get(m), cur_agg.get(m)
294                 if b is None or c is None:
295                     continue
296                 if c > b + tols.rate_tol:
297                     cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
298                 elif c < b - tols.rate_tol:
299                     cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
300
301         # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
302         for vid in sorted(base_names & cur_names):
303             for m in PER_VIDEO_GUARD:
304                 b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
305                 if b is None or c is None:
306                     continue
307                 if c > b + tols.over_seg_tol:
308                     cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))
309
310         if cmp.regressions:
311             cmp.status = "regression"
312         elif cmp.status != "stale":
313             cmp.status = "ok"
314         return cmp
315
316
317     @dataclass
318     class NightlyResult:
319         comparisons: List[ConfigComparison] = field(default_factory=list)
320         skipped: Dict[str, List[str]] = field(default_factory=dict)  # config -> manifest
videos not scored
321
322         @property
323         def has_regression(self) -> bool:
324             return any(c.status == "regression" for c in self.comparisons)
325
326         @property
327         def has_stale(self) -> bool:
328             return any(c.status == "stale" for c in self.comparisons)
329
330         def overall_status(self) -> str:
331             if self.has_regression:
332                 return "REGRESSION"
333             if self.has_stale:
334                 return "STALE"
```

```
            return "PASS"

        def exit_code(self) -> int:
            if self.has_regression:
                return 1
            if self.has_stale:
                return 4
            return 0


    def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
                    tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
        """Compare every tracked config's current summary to the baseline summary."""
        res = NightlyResult(skipped=skipped or {})
        base_cfgs = baseline.get("configs", {})
        cur_cfgs = current.get("configs", {})
        for name in cur_cfgs:
            if name not in base_cfgs:
                cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
                cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
                res.comparisons.append(cmp)
                continue
            res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
        return res


    # ---------------------------------------------------------------------- #
    # Report formatting (pure).
    # ---------------------------------------------------------------------- #
    def _fmt(v: Optional[float]) -> str:
        return "?" if v is None else f"{v:.4f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, tols: Tolerances, date: str) -> str:
        L: List[str] = []
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
        L.append(f"# Nightly rally-quality regression ? {date}")
        L.append("")
        L.append(f"**Status: {badge}**  ·  exit code {result.exit_code()}")
        L.append("")
        L.append(f"- HEAD: `{current.get('git_commit', '?')}`  ·  baseline: "
                 f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
        L.append(f"- Corpus manifest: `{current.get('manifest', '?')}`  ·
IoU={current.get('iou', DEFAULT_IOU)}")
        L.append(f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
                 f"per-video over-seg +{tols.over_seg_tol}")
        L.append("")

        # Up-front regression / stale summary.
        regs = [f for c in result.comparisons for f in c.regressions]
        if regs:
            L.append("## ? Regressions")
            L.append("")
            for f in regs:
                L.append(f"- {f.message()}")
            L.append("")
        if result.has_stale:
            L.append("## ? Stale / not comparable")
```

```
            L.append("")
            for c in result.comparisons:
                if c.status == "stale":
                    if c.added_videos:
                        L.append(f"- `{c.config}`: corpus added {c.added_videos}")
                    if c.removed_videos:
                        L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
                    if not c.added_videos and not c.removed_videos:
                        L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
            L.append("- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
            L.append("")

        # Skipped (no silent caps).
        skipped_any = {k: v for k, v in result.skipped.items() if v}
        if skipped_any:
            L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
            L.append("")
            for cfg, vids in skipped_any.items():
                L.append(f"- `{cfg}`: {vids}")
            L.append("")

        # Per-config detail.
        cur_cfgs = current.get("configs", {})
        base_cfgs = baseline.get("configs", {})
        for cmp in result.comparisons:
            cur = cur_cfgs.get(cmp.config, {})
            base = base_cfgs.get(cmp.config, {})
            L.append(f"## Config `{cmp.config}` ? {cur.get('note', '')}")
            L.append("")
            L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})_")
            L.append("")
            cagg = cur.get("aggregate", {})
            bagg = base.get("aggregate", {})
            L.append("| metric | baseline | current | ? | gated |")
            L.append("|---|---|---|---|---|")
            for m in AGG_SNAPSHOT:
                b, c = bagg.get(m), cagg.get(m)
                d = (c - b) if (b is not None and c is not None) else None
                gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
                L.append(f"| {m} | {_fmt(b)} | {_fmt(c)} | {_fmt(d)} | {gated} |")
            L.append("")
            # Per-video over-seg guard table.
            cpv = cur.get("per_video", {})
            bpv = base.get("per_video", {})
            names = sorted(set(cpv) | set(bpv))
            if names:
                L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
                L.append("|---|---|---|---|")
                for v in names:
                    cb, cc = bpv.get(v, {}), cpv.get(v, {})
                    tf = f"{_fmt(cb.get('tol_f1'))}?{_fmt(cc.get('tol_f1'))}"
                    sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
                    mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"
                    flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
                    L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
                L.append("")
            imps = cmp.improvements
            if imps:
                L.append("_Improvements over baseline (consider `--update-baseline` to
```

```
ratchet the floor up):_")
                for f in imps:
                    L.append(f"- {f.message()}")
                L.append("")

        L.append("---")
        L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change "
                 "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
        return "\n".join(L)


    # ------------------------------------------------------------------------ #
    # I/O shell.
    # ------------------------------------------------------------------------ #
    def _git_commit() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:
            return "?"


    def _display_path(path: str) -> str:
        """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
        in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
        Metadata only ? the comparison never reads it."""
        norm = path.replace("\\", "/")
        parent = os.path.basename(os.path.dirname(norm))
        if parent == "output":
            return "output/" + os.path.basename(norm)
        try:
            rel = os.path.relpath(path, REPO_ROOT)
            if not rel.startswith(".."):
                return rel.replace("\\", "/")
        except ValueError:
            pass
        return norm


    def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
        """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).

        ``snapshot`` is the serialisable structure stored as / compared to the baseline.
``skipped``
        lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
surfaced in
        the report so the corpus can't silently shrink."""
        from backend.eval import rally_seg_eval

        golden = rally_seg_eval.load_golden(manifest, features_dir)
        manifest_names = [v["name"] for v in golden]
        if not golden:
            raise RuntimeError(f"no golden videos loaded from {manifest}")

        snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
"configs": {}}
        skipped: Dict[str, List[str]] = {}
```

```
        scored_any = False
        for cfg in configs:
            result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
min_crossings=cfg.min_crossings)
            summary = summarize_run(result)
            summary["note"] = cfg.note
            snapshot["configs"][cfg.name] = summary
            scored = set(summary["per_video"])
            skipped[cfg.name] = [n for n in manifest_names if n not in scored]
            scored_any = scored_any or bool(scored)
        if not scored_any:
            raise RuntimeError(
                f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
                f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?")
        return snapshot, skipped


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_commit()
        out["tolerances"] = tols.to_dict()
        out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
                       "`python scripts/nightly_regression.py --update-baseline` after an
intended "
                       "improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed).")
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
(offline, CPU).")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
        p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
                       help="dir holding <name>_golden_features.json")
        p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
path")
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
are written")
        p.add_argument("--iou", type=float, default=DEFAULT_IOU)
        p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
                       help="F1-like aggregate drop that counts as a regression")
        p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
                       help="merge_rate rise that counts as a regression")
        p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
                       help="per-video split/merge-count increase allowed before it's a
regression")
        p.add_argument("--update-baseline", action="store_true",
                       help="snapshot the current numbers as the new baseline instead of
checking")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
```

```
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_regression")
        args = build_parser().parse_args(argv)
        tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
over_seg_tol=args.over_seg_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s ? the golden corpus must be present locally.",
args.manifest)
            return 2

        try:
            snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
args.features_dir, args.iou)
        except RuntimeError as e:
            log.error("%s", e)
            return 2
        snapshot["git_commit"] = _git_commit()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot, tols)
            log.info("baseline snapshot written to %s", args.baseline)
            for cfg, s in snapshot["configs"].items():
                agg = s["aggregate"]
                log.info("  %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
                         cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
                         agg.get("merge_rate") or 0.0)
            return 0

        baseline = load_baseline(args.baseline)
        if baseline is None:
            log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
            return 3

        result = compare_all(baseline, snapshot, tols, skipped)
        report = format_report(baseline, snapshot, result, tols, date)

        if not args.no_report:
            os.makedirs(args.report_dir, exist_ok=True)
            md_path = os.path.join(args.report_dir, f"{date}.md")
            json_path = os.path.join(args.report_dir, f"{date}.json")
            with open(md_path, "w", encoding="utf-8") as fh:
                fh.write(report + "\n")
            with open(json_path, "w", encoding="utf-8") as fh:
                json.dump({"date": date, "status": result.overall_status(),
                           "exit_code": result.exit_code(), "current": snapshot,
                           "regressions": [f.message() for c in result.comparisons for f in
c.regressions]},
                          fh, indent=2, sort_keys=True)
            log.info("report written to %s", md_path)

        print(report)
        code = result.exit_code()
        if code == 1:
```

```
623             log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                       sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625         elif code == 4:
626             log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627         else:
628             log.info("nightly regression check PASSED")
629         return code
630
631
632     if __name__ == "__main__":
633         raise SystemExit(main())
634
```

34. assistant (2026-06-22T09:13:08.582Z)

Now let me check the baseline file to understand the actual signal being tracked:

35. user (2026-06-22T09:13:10.179Z)


```
1       {
2         "_doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3         "configs": {
4           "baseline_off": {
5             "aggregate": {
6               "dend_abs_median": 41.77921047714383,
7               "dstart_abs_median": 42.467243955066174,
8               "f1@0.25": 0.39981006941991765,
9               "f1@0.5": 0.2366145803119009,
10              "merge_rate": 0.6380248744753217,
11              "precision": 0.2577557018251538,
12              "recall": 0.22719343607869802,
13              "segment_count_ratio": 0.5824947574930804,
14              "split_rate": 0.0,
15              "tol_f1": 0.1382064040388172,
16              "tol_precision": 0.14536686157033415,
17              "tol_recall": 0.13445461315285676
18            },
19            "iou": 0.5,
20            "min_crossings": 0,
21            "n": 15,
22            "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",
23            "per_video": {
24              "Badminton_BXH_2": {
25                "f1@0.25": 0.0,
26                "f1@0.5": 0.0,
27                "merge_count": 3,
28                "merge_rate": 1.0,
29                "num_gt": 44,
30                "num_pred": 4,
31                "split_count": 0,
32                "tol_f1": 0.0
33              },
34              "Boxhill_Doubles": {
35                "f1@0.25": 0.10526315789473685,
36                "f1@0.5": 0.03508771929824561,
37                "merge_count": 7,
38                "merge_rate": 0.9534883720930233,
39                "num_gt": 43,
40                "num_pred": 14,
41                "split_count": 0,
42                "tol_f1": 0.0
```

```
    },
    "GX010128": {
      "f1@0.25": 0.14925373134328357,
      "f1@0.5": 0.05970149253731343,
      "merge_count": 12,
      "merge_rate": 0.9615384615384616,
      "num_gt": 52,
      "num_pred": 15,
      "split_count": 0,
      "tol_f1": 0.0
    },
    "GX010137": {
      "f1@0.25": 0.9253731343283583,
      "f1@0.5": 0.5970149253731343,
      "merge_count": 3,
      "merge_rate": 0.17647058823529413,
      "num_gt": 34,
      "num_pred": 33,
      "split_count": 0,
      "tol_f1": 0.4477611940298507
    },
    "GX010141": {
      "f1@0.25": 0.5416666666666666,
      "f1@0.5": 0.3333333333333333,
      "merge_count": 5,
      "merge_rate": 0.6551724137931034,
      "num_gt": 29,
      "num_pred": 19,
      "split_count": 0,
      "tol_f1": 0.3333333333333333
    },
    "GX020094": {
      "f1@0.25": 0.7333333333333334,
      "f1@0.5": 0.4444444444444445,
      "merge_count": 3,
      "merge_rate": 0.15,
      "num_gt": 40,
      "num_pred": 50,
      "split_count": 0,
      "tol_f1": 0.2888888888888889
    },
    "GX030094": {
      "f1@0.25": 0.5352112676056339,
      "f1@0.5": 0.28169014084507044,
      "merge_count": 8,
      "merge_rate": 0.5609756097560976,
      "num_gt": 41,
      "num_pred": 30,
      "split_count": 0,
      "tol_f1": 0.14084507042253522
    },
    "adarsh_avi_singles": {
      "f1@0.25": 0.8601036269430052,
      "f1@0.5": 0.6321243523316062,
      "merge_count": 5,
      "merge_rate": 0.10416666666666667,
      "num_gt": 96,
      "num_pred": 97,
      "split_count": 0,
      "tol_f1": 0.38341968911917096
    },
    "gbaaddy": {
      "f1@0.25": 0.17142857142857143,
      "f1@0.5": 0.08571428571428572,
      "merge_count": 9,
```

```
      "merge_rate": 0.8541666666666666,
      "num_gt": 48,
      "num_pred": 22,
      "split_count": 0,
      "tol_f1": 0.0
    },
    "kushagra_singles": {
      "f1@0.25": 0.0,
      "f1@0.5": 0.0,
      "merge_count": 2,
      "merge_rate": 1.0,
      "num_gt": 32,
      "num_pred": 2,
      "split_count": 0,
      "tol_f1": 0.0
    },
    "largetest_doubles": {
      "f1@0.25": 0.8118811881188118,
      "f1@0.5": 0.5346534653465348,
      "merge_count": 4,
      "merge_rate": 0.16326530612244897,
      "num_gt": 49,
      "num_pred": 52,
      "split_count": 0,
      "tol_f1": 0.297029702970297
    },
    "mahadevpura_1": {
      "f1@0.25": 0.0,
      "f1@0.5": 0.0,
      "merge_count": 1,
      "merge_rate": 1.0,
      "num_gt": 10,
      "num_pred": 1,
      "split_count": 0,
      "tol_f1": 0.0
    },
    "mahadevpura_2": {
      "f1@0.25": 0.0,
      "f1@0.5": 0.0,
      "merge_count": 3,
      "merge_rate": 0.975,
      "num_gt": 40,
      "num_pred": 5,
      "split_count": 0,
      "tol_f1": 0.0
    },
    "mahadevpura_singles": {
      "f1@0.25": 0.6181818181818182,
      "f1@0.5": 0.3636363636363636,
      "merge_count": 6,
      "merge_rate": 0.5161290322580645,
      "num_gt": 31,
      "num_pred": 24,
      "split_count": 0,
      "tol_f1": 0.1818181818181818
    },
    "testlarge_short": {
      "f1@0.25": 0.5454545454545454,
      "f1@0.5": 0.1818181818181818,
      "merge_count": 1,
      "merge_rate": 0.5,
      "num_gt": 6,
      "num_pred": 5,
      "split_count": 0,
      "tol_f1": 0.0
```

```
            }
          },
          "preset": {
            "blob_factor": 4.0,
            "blob_fallback": false,
            "hard_cap": false,
            "inactivity_end": 0.0,
            "inactivity_min_density": 0.34,
            "inactivity_rally_thresh": 0.08,
            "max_window_duration": 0.0,
            "merge_gap": 2.0,
            "min_window_duration": 2.0,
            "name": "baseline_off",
            "velocity_thresh": 0.02
          }
        },
        "default": {
          "aggregate": {
            "dend_abs_median": 2.9720372594817066,
            "dstart_abs_median": 3.277048437326215,
            "f1@0.25": 0.6296769405815345,
            "f1@0.5": 0.4681709925781376,
            "merge_rate": 0.13955677464298155,
            "precision": 0.39945838409296397,
            "recall": 0.5917335962330964,
            "segment_count_ratio": 1.4784540509868136,
            "split_rate": 0.11823489673413166,
            "tol_f1": 0.3532006050209596,
            "tol_precision": 0.30232209566271007,
            "tol_recall": 0.44486191614359216
          },
          "iou": 0.5,
          "min_crossings": 0,
          "n": 15,
          "note": "SERVED production windowing \u2014 now the R1 milestone (cap=6 + R4)
after the #204 flip.",
          "per_video": {
            "Badminton_BXH_2": {
              "f1@0.25": 0.6721311475409836,
              "f1@0.5": 0.4426229508196722,
              "merge_count": 2,
              "merge_rate": 0.09090909090909091,
              "num_gt": 44,
              "num_pred": 78,
              "split_count": 4,
              "tol_f1": 0.3278688524590163
            },
            "Boxhill_Doubles": {
              "f1@0.25": 0.5538461538461538,
              "f1@0.5": 0.4307692307692308,
              "merge_count": 0,
              "merge_rate": 0.0,
              "num_gt": 43,
              "num_pred": 87,
              "split_count": 9,
              "tol_f1": 0.26153846153846155
            },
            "GX010128": {
              "f1@0.25": 0.7218045112781956,
              "f1@0.5": 0.6015037593984963,
              "merge_count": 0,
              "merge_rate": 0.0,
              "num_gt": 52,
              "num_pred": 81,
              "split_count": 2,
```

```
      "tol_f1": 0.46616541353383456
    },
    "GX010137": {
      "f1@0.25": 0.9041095890410958,
      "f1@0.5": 0.8219178082191781,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 34,
      "num_pred": 39,
      "split_count": 1,
      "tol_f1": 0.6849315068493151
    },
    "GX010141": {
      "f1@0.25": 0.7450980392156864,
      "f1@0.5": 0.5098039215686274,
      "merge_count": 6,
      "merge_rate": 0.4482758620689655,
      "num_gt": 29,
      "num_pred": 22,
      "split_count": 0,
      "tol_f1": 0.4313725490196078
    },
    "GX020094": {
      "f1@0.25": 0.6666666666666665,
      "f1@0.5": 0.4242424242424242,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 40,
      "num_pred": 59,
      "split_count": 7,
      "tol_f1": 0.36363636363636365
    },
    "GX030094": {
      "f1@0.25": 0.6451612903225806,
      "f1@0.5": 0.4677419354838709,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 41,
      "num_pred": 83,
      "split_count": 6,
      "tol_f1": 0.41935483870967744
    },
    "adarsh_avi_singles": {
      "f1@0.25": 0.7873303167420814,
      "f1@0.5": 0.6787330316742081,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 96,
      "num_pred": 125,
      "split_count": 13,
      "tol_f1": 0.5520361990950227
    },
    "gbaaddy": {
      "f1@0.25": 0.638888888888889,
      "f1@0.5": 0.4166666666666667,
      "merge_count": 1,
      "merge_rate": 0.041666666666666664,
      "num_gt": 48,
      "num_pred": 96,
      "split_count": 8,
      "tol_f1": 0.2777777777777778
    },
    "kushagra_singles": {
      "f1@0.25": 0.6753246753246752,
      "f1@0.5": 0.41558441558441556,
```

```
      "merge_count": 3,
      "merge_rate": 0.1875,
      "num_gt": 32,
      "num_pred": 45,
      "split_count": 6,
      "tol_f1": 0.07792207792207793
    },
    "largetest_doubles": {
      "f1@0.25": 0.8,
      "f1@0.5": 0.6666666666666666,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 49,
      "num_pred": 71,
      "split_count": 5,
      "tol_f1": 0.5833333333333334
    },
    "mahadevpura_1": {
      "f1@0.25": 0.0,
      "f1@0.5": 0.0,
      "merge_count": 2,
      "merge_rate": 0.8,
      "num_gt": 10,
      "num_pred": 2,
      "split_count": 0,
      "tol_f1": 0.0
    },
    "mahadevpura_2": {
      "f1@0.25": 0.4878048780487805,
      "f1@0.5": 0.24390243902439024,
      "merge_count": 9,
      "merge_rate": 0.525,
      "num_gt": 40,
      "num_pred": 42,
      "split_count": 4,
      "tol_f1": 0.12195121951219512
    },
    "mahadevpura_singles": {
      "f1@0.25": 0.7469879518072289,
      "f1@0.5": 0.6024096385542168,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 31,
      "num_pred": 52,
      "split_count": 7,
      "tol_f1": 0.5301204819277108
    },
    "testlarge_short": {
      "f1@0.25": 0.4,
      "f1@0.5": 0.3,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 6,
      "num_pred": 14,
      "split_count": 1,
      "tol_f1": 0.2
    }
  },
  "preset": {
    "blob_factor": 4.0,
    "blob_fallback": false,
    "hard_cap": false,
    "inactivity_end": 1.5,
    "inactivity_min_density": 0.3,
    "inactivity_rally_thresh": 0.2,
```

```
            "max_window_duration": 6.0,
            "merge_gap": 2.0,
            "min_window_duration": 2.0,
            "name": "served",
            "velocity_thresh": 0.02
          }
        },
        "milestone": {
          "aggregate": {
            "dend_abs_median": 2.9720372594817066,
            "dstart_abs_median": 3.277048437326215,
            "f1@0.25": 0.6296769405815345,
            "f1@0.5": 0.4681709925781376,
            "merge_rate": 0.13955677464298155,
            "precision": 0.39945838409296397,
            "recall": 0.5917335962330964,
            "segment_count_ratio": 1.4784540509868136,
            "split_rate": 0.11823489673413166,
            "tol_f1": 0.3532006050209596,
            "tol_precision": 0.30232209566271007,
            "tol_recall": 0.44486191614359216
          },
          "iou": 0.5,
          "min_crossings": 0,
          "n": 15,
          "note": "R3 cap=6 s + R4 rally-END inactivity trim \u2014 the shipped milestone
(== default post-#204).",
          "per_video": {
            "Badminton_BXH_2": {
              "f1@0.25": 0.6721311475409836,
              "f1@0.5": 0.4426229508196722,
              "merge_count": 2,
              "merge_rate": 0.09090909090909091,
              "num_gt": 44,
              "num_pred": 78,
              "split_count": 4,
              "tol_f1": 0.3278688524590163
            },
            "Boxhill_Doubles": {
              "f1@0.25": 0.5538461538461538,
              "f1@0.5": 0.4307692307692308,
              "merge_count": 0,
              "merge_rate": 0.0,
              "num_gt": 43,
              "num_pred": 87,
              "split_count": 9,
              "tol_f1": 0.26153846153846155
            },
            "GX010128": {
              "f1@0.25": 0.7218045112781956,
              "f1@0.5": 0.6015037593984963,
              "merge_count": 0,
              "merge_rate": 0.0,
              "num_gt": 52,
              "num_pred": 81,
              "split_count": 2,
              "tol_f1": 0.46616541353383456
            },
            "GX010137": {
              "f1@0.25": 0.9041095890410958,
              "f1@0.5": 0.8219178082191781,
              "merge_count": 0,
              "merge_rate": 0.0,
              "num_gt": 34,
              "num_pred": 39,
```

```
      "split_count": 1,
      "tol_f1": 0.6849315068493151
    },
    "GX010141": {
      "f1@0.25": 0.7450980392156864,
      "f1@0.5": 0.5098039215686274,
      "merge_count": 6,
      "merge_rate": 0.4482758620689655,
      "num_gt": 29,
      "num_pred": 22,
      "split_count": 0,
      "tol_f1": 0.4313725490196078
    },
    "GX020094": {
      "f1@0.25": 0.6666666666666665,
      "f1@0.5": 0.4242424242424242,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 40,
      "num_pred": 59,
      "split_count": 7,
      "tol_f1": 0.36363636363636365
    },
    "GX030094": {
      "f1@0.25": 0.6451612903225806,
      "f1@0.5": 0.4677419354838709,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 41,
      "num_pred": 83,
      "split_count": 6,
      "tol_f1": 0.41935483870967744
    },
    "adarsh_avi_singles": {
      "f1@0.25": 0.7873303167420814,
      "f1@0.5": 0.6787330316742081,
      "merge_count": 0,
      "merge_rate": 0.0,
      "num_gt": 96,
      "num_pred": 125,
      "split_count": 13,
      "tol_f1": 0.5520361990950227
    },
    "gbaaddy": {
      "f1@0.25": 0.638888888888889,
      "f1@0.5": 0.4166666666666667,
      "merge_count": 1,
      "merge_rate": 0.041666666666666664,
      "num_gt": 48,
      "num_pred": 96,
      "split_count": 8,
      "tol_f1": 0.2777777777777778
    },
    "kushagra_singles": {
      "f1@0.25": 0.6753246753246752,
      "f1@0.5": 0.41558441558441556,
      "merge_count": 3,
      "merge_rate": 0.1875,
      "num_gt": 32,
      "num_pred": 45,
      "split_count": 6,
      "tol_f1": 0.07792207792207793
    },
    "largetest_doubles": {
      "f1@0.25": 0.8,
```

```
            "f1@0.5": 0.6666666666666666,
            "merge_count": 0,
            "merge_rate": 0.0,
            "num_gt": 49,
            "num_pred": 71,
            "split_count": 5,
            "tol_f1": 0.5833333333333334
          },
          "mahadevpura_1": {
            "f1@0.25": 0.0,
            "f1@0.5": 0.0,
            "merge_count": 2,
            "merge_rate": 0.8,
            "num_gt": 10,
            "num_pred": 2,
            "split_count": 0,
            "tol_f1": 0.0
          },
          "mahadevpura_2": {
            "f1@0.25": 0.4878048780487805,
            "f1@0.5": 0.24390243902439024,
            "merge_count": 9,
            "merge_rate": 0.525,
            "num_gt": 40,
            "num_pred": 42,
            "split_count": 4,
            "tol_f1": 0.12195121951219512
          },
          "mahadevpura_singles": {
            "f1@0.25": 0.7469879518072289,
            "f1@0.5": 0.6024096385542168,
            "merge_count": 0,
            "merge_rate": 0.0,
            "num_gt": 31,
            "num_pred": 52,
            "split_count": 7,
            "tol_f1": 0.5301204819277108
          },
          "testlarge_short": {
            "f1@0.25": 0.4,
            "f1@0.5": 0.3,
            "merge_count": 0,
            "merge_rate": 0.0,
            "num_gt": 6,
            "num_pred": 14,
            "split_count": 1,
            "tol_f1": 0.2
          }
        },
        "preset": {
          "blob_factor": 4.0,
          "blob_fallback": false,
          "hard_cap": false,
          "inactivity_end": 1.5,
          "inactivity_min_density": 0.3,
          "inactivity_rally_thresh": 0.2,
          "max_window_duration": 6.0,
          "merge_gap": 2.0,
          "min_window_duration": 2.0,
          "name": "milestone",
          "velocity_thresh": 0.02
        }
      }
    },
    "generated_at": "2026-06-19",
```

```
561       "git_commit": "a240221",
562       "iou": 0.5,
563       "manifest": "output/human_lovo_manifest.json",
564       "tolerances": {
565         "metric_tol": 0.005,
566         "over_seg_tol": 0,
567         "rate_tol": 0.01
568       }
569     }
570
```

36. assistant (2026-06-22T09:13:12.526Z)

Now I have a critical finding. Let me verify by looking at what "reel" actually means in the context. Let me search for reel output files:

37. user (2026-06-22T09:13:14.132Z)

```
    # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel watermark
def _reels_dir() -> str:
_REEL_EXTS = (".mp4", ".mov")
@app.get("/api/reels")
def list_reels():
    """List compiled highlight reels present in the output dir, newest first, with a download
URL.
    NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
    does NOT track a video_id?reel association today (a per-video listing would be a phantom),
so
    this is a global list. (Per-video reel tracking would need a DB record + compile change ? a
    base = _reels_dir()
    reels = []
        return {"reels": []}
                reels.append({
                    "download_url": f"/api/reels/{quote(e.name)}",
    reels.sort(key=lambda r: r["modified_at"], reverse=True)
    return {"reels": reels}
@app.get("/api/reels/{filename}")
def download_reel(filename: str):
    """Stream a compiled reel by its bare filename (no video_id ? reels are flat in output_dir).
    base = _reels_dir()
        raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
    # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the user's
language
        out_filename = args.trim_output or "trimmed_segment.mp4"
```

38. assistant (2026-06-22T09:13:16.564Z)

Perfect! Now I understand the full picture. Let me verify the key claim one more time by checking the actual API endpoint and what it counts:

39. user (2026-06-22T09:13:17.452Z)

```
381     def _reels_dir() -> str:
382         return getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
383
384
385     _REEL_EXTS = (".mp4", ".mov")
386
387
388     @app.get("/api/reels")
389     def list_reels():
```

```
390         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
391
392         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
393         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
394         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
395         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
396         base = _reels_dir()
397         reels = []
398         try:
399             entries = os.scandir(base)
400         except OSError:
401             return {"reels": []}
402         with entries:
403             for e in entries:
404                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
405                     try:
406                         st = e.stat()
407                     except OSError:
408                         continue
409                     reels.append({
410                         "filename": e.name,
411                         "size_bytes": st.st_size,
412                         "modified_at": st.st_mtime,
413                         "download_url": f"/api/reels/{quote(e.name)}",
414                     })
415         reels.sort(key=lambda r: r["modified_at"], reverse=True)
416         return {"reels": reels}
417
418
419     @app.get("/api/reels/{filename}")
420     def download_reel(filename: str):
421         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
422         Mirrors /api/highlights' safety (bare-name + containment checks)."""
423         try:
424             name = safe_output_filename(filename)
425         except ValueError as e:
426             raise HTTPException(status_code=400, detail=str(e)) from e
427         base = _reels_dir()
428         path = os.path.join(base, name)
429         try:
430             path = resolve_under_root(path, base)
431         except ValueError as e:
432             raise HTTPException(status_code=400, detail=str(e)) from e
433         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
434             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
435         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
436         return FileResponse(path, media_type=media, filename=name)
437
438
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
440         """The full hash ? validate ? segment ? report pipeline for one video.
```

40. assistant (2026-06-22T09:13:19.844Z)

Now let me verify the context from the mapping phase was about "reel" meaning actual output files. Let me check if there's any simple tracking mechanism:

41. user (2026-06-22T09:13:29.771Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/config/models.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/infrastructure/database.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/gemini.py

42. assistant (2026-06-22T09:13:31.685Z)

Let me check database.py for any reel-related counting:

43. user (2026-06-22T09:13:32.562Z)

```
400:        # count_jobs_by_locale) and pairs with the localized reel watermark (the SPA sends the same
```

44. assistant (2026-06-22T09:13:38.555Z)

That's just a comment. Now let me analyze the actual regression test structure. The key insight is that the context claims the regression signal is "COUNT OF PLAY_SEGMENTS ROWS" but I need to verify this against what the nightly_regression.py actually does:

Looking at the nightly_regression.py, I can see it's evaluating RALLY DETECTION QUALITY (F1 scores, merge rates, etc.) across 15 golden videos using a pre-cached trajectory dataset. It's NOT counting reels or play_segments directly. The regression is on **metrics** like `tol_f1`, `f1@0.5`, `merge_rate` ? which measure how well the rally segmentation matches ground truth.

This is completely different from the claim in the context which says the regression signal is "COUNT OF PLAY_SEGMENTS ROWS in the SQLite database."

Let me verify this is correct by checking what the actual nightly pipeline does:

45. user (2026-06-22T09:13:39.363Z)

```
1       <#
2       .SYNOPSIS
3           Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5       .DESCRIPTION
6           Re-scores the golden corpus under the current checkout via scripts/nightly_regression.py
7           (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline. Writes a
8           dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker, then exits
9           with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11          GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present yet (e.g.
12          before this PR is merged into the checkout the task points at) ? so a registered task never
13          errors in the interim and self-activates once the files land.
14
15          Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE the repo on
16          this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18      .PARAMETER RepoRoot
19          Repo root to test. Default: the parent of this script's directory.
20
21      .PARAMETER PythonExe
22          Python interpreter. Default: 'python'.
23
24      .PARAMETER Pull
```

```
25          If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26          that may hold a sibling session's WIP).
27      #>
28      [CmdletBinding()]
29      param(
30          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31          [string]$PythonExe = 'python',
32          [switch]$Pull
33      )
34
35      $ErrorActionPreference = 'Continue'
36      $date = Get-Date -Format 'yyyy-MM-dd'
37      $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38      $logDir = Join-Path $reportDir 'logs'
39      New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40      $log = Join-Path $logDir "$date.log"
41      $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
42
43      function Write-Status([string]$status, [int]$code, [string]$detail) {
44          $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45          "$stamp  status=$status exit=$code  $detail" | Out-File -FilePath $statusFile
-Encoding utf8
46      }
47
48      $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49      $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
50
51      # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
52      if (-not (Test-Path $checker)) {
53          "[$date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
| Tee-Object -FilePath $log
54          Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
55          exit 0
56      }
57      if (-not (Test-Path $manifest)) {
58          "[$date] golden manifest not present at $manifest ? skipping (corpus absent on this
machine)." | Tee-Object -FilePath $log
59          Write-Status 'SKIPPED' 0 'golden corpus absent'
60          exit 0
61      }
62
63      Set-Location $RepoRoot
64      if ($Pull) {
65          "[$date] git pull --ff-only" | Tee-Object -FilePath $log -Append
66          & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
67      }
68
69      $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
70      "[$date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
-Append
71
72      & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
73      $code = $LASTEXITCODE
74
75      switch ($code) {
76          0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
77          1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
78              "[$date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
Object -FilePath $log -Append }
79          4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
80          default { Write-Status 'ERROR' $code "checker error ? see $log" }
81      }
82
```

```
83      exit $code
84
```

46. assistant (2026-06-22T09:13:42.105Z)

Now I have the complete picture. The nightly regression is NOT about counting reels or play_segments. Let me verify this one more time by checking if there's any OTHER nightly test that might be about reel counting:

47. user (2026-06-22T09:13:43.842Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/eval_baselines/nightly_baseline.json
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/run_regression.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.ps1
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/register_nightly_task.ps1
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/tests/test_nightly_regression.py
```

48. assistant (2026-06-22T09:13:45.503Z)

Let me check run_regression.py:

49. user (2026-06-22T09:13:46.286Z)

```
1       """CLI: regression-test the production tool against golden-set ground truth.
2
3       Obtains ground truth from an annotation provider (a tier), scores the video's
4       stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5       detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7       Examples
8       --------
9       Snapshot the current numbers as the baseline (first run / after a real improvement):
10          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12      Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14      """
15      import os
16      import sys
17      import json
18      import argparse
19      import logging
20
21      from backend.infrastructure.database import Database
22      from backend.config import load_config
23      from backend.annotations import annotation_registry
24      from backend.eval import regression
25
26
27      def _default_db_path() -> str:
28          cfg = load_config()
29          return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32      def build_parser() -> argparse.ArgumentParser:
33          p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
```

```
ground truth.")
34          p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35          p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37          p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38          p.add_argument("--stage", choices=("candidates", "final"), default="final")
39          p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40          p.add_argument("--iou", type=float, default=0.5)
41          p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42          p.add_argument("--db", default=None)
43          p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44          p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46          p.add_argument("--allow-missing-baseline", action="store_true",
47                         help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48          p.add_argument("--json", dest="json_out", default=None)
49          return p
50
51
52      def main(argv=None) -> int:
53          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54          args = build_parser().parse_args(argv)
55
56          db_path = args.db or _default_db_path()
57          if not os.path.exists(db_path):
58              print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59              return 2
60          db = Database(db_path)
61
62          result = regression.regress_with_provider(
63              db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64              stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65              tolerance=args.tolerance, baseline_path=args.baseline,
66          )
67
68          print(regression.format_result(result))
69
70          if args.update_baseline:
71              regression.save_baseline(result.video_id, result.stage, result.precision,
72                                       result.recall, result.f1, path=args.baseline,
73                                       iou_threshold=result.iou_threshold,
detector=args.detector,
74                                       gt_fingerprint=result.gt_hash)
75              print(f"Baseline updated in {args.baseline}.")
76
77          if args.json_out:
78              with open(args.json_out, "w") as f:
79                  json.dump(result.as_dict(), f, indent=2)
80              print(f"JSON written to {args.json_out}")
81
82          # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83          if result.regressed:
84              return 1
85          if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86              print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
```

```
87                    "Snapshot one with --update-baseline (then commit it), or pass "
88                    "--allow-missing-baseline to accept this run.", file=sys.stderr)
89                return 3
90            return 0
91
92
93        if __name__ == "__main__":
94            raise SystemExit(main())
95
```

50. assistant (2026-06-22T09:13:49.607Z)

This is ALSO not about counting reels. It's about regression testing precision/recall. Now let me check the CLAUDE.md memory to see if there's more context:

Based on my thorough adversarial verification, I can now provide the structured result. The claim in the context is definitively REFUTED by the actual code evidence.

51. user (2026-06-22T09:14:02.429Z)

Structured output provided successfully